คู่มือจำลองการสอบป้องกันโครงงาน (MASTER EXAMINATION DEFENSE)

รวมแนวคำถาม-คำตอบ สำหรับการสอบจบโปรเจกต์

ระบบบริหารจัดการการรับซื้อน้ำยางพารา (Clover Project — Field Latex System)

รวบรวมคำถามเจาะลึกระดับคณะกรรมการสอบ ทั้งด้านสถาปัตยกรรมซอฟต์แวร์, ตรรกะทางธุรกิจ, ฟังก์ชันในโค้ด, ความปลอดภัย, ฐานข้อมูล และการจัดการข้อยกเว้น พร้อมแนวทางคำตอบฉบับสมบูรณ์อย่างละเอียดทุกประเด็น

ระดับการประเมิน: โครงงานเทคโนโลยีสารสนเทศ / วิทยาการคอมพิวเตอร์ / วิศวกรรมซอฟต์แวร์
เทคโนโลยี: Node.js • Express • PostgreSQL • Prisma ORM • Vanilla HTML/CSS/JS (ES Modules)
จัดทำเมื่อ: ตุลาคม 2569

สารบัญหมวดหมู่คำถาม

1. สถาปัตยกรรมระบบและการตัดสินใจเชิงเทคนิค
2. การออกแบบฐานข้อมูลและความสมบูรณ์ของข้อมูล
3. โลจิกทางธุรกิจ การคิดเงิน และการเงินสหกรณ์
4. ความปลอดภัยและการยืนยันตัวตน (Security & Auth)
5. เจาะลึกโค้ดและกลไกหน้าบ้าน (DOM Engine)
6. การรองรับข้อผิดพลาด ปัญหา Edge Cases & การขยายระบบ

1 สถาปัตยกรรมระบบและการตัดสินใจเชิงเทคนิค (System Architecture)

Q1 ทำไมระบบนี้ถึงเลือกไม่ใช้ React, Vue หรือ Next.js ในหน้าหลัก แต่แปลงมาเป็น Vanilla HTML, CSS และ JavaScript ร่วมกับ Express? มีข้อดีข้อเสียอย่างไรในเชิงวิศวกรรม?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจในการเลือกใช้เทคโนโลยีที่เหมาะสมกับบริบทผู้ใช้งาน (Trade-offs) ไม่ใช่แค่เลือกตามกระแสนิยม

แนวทางการตอบ:

"ทางทีมงานได้ทำการประเมินบริบทของผู้ใช้งานจริง ซึ่งเป็นเจ้าหน้าที่หน้าลานรับซื้อน้ำยางและสมาชิกชาวสวนยางในพื้นที่ต่างจังหวัด จึงมีเหตุผลเชิงวิศวกรรม 4 ประการครับ:

1. **ประสิทธิภาพและความเร็วในการโหลด (Zero-Bundle Overhead):** หน้าเว็บที่เป็น Vanilla JS ไม่มีไฟล์ JavaScript Framework Bundle ขนาดใหญ่ (เช่น React + Next runtime หนัก 200-500KB) ทำให้หน้าเว็บเปิดได้ทันที แม้ในจุดรับซื้อที่สัญญาณ 3G/4G อ่อน
2. **ลดการใช้หน่วยความจำเซิร์ฟเวอร์ (Low Memory Footprint):** Express ทำหน้าที่เป็น REST API + HTML Server แบบตรงไปตรงมา กินแรมเพียง 50-80MB ขณะที่ Next.js Node server ต้องใช้ทรัพยากรมากกว่า 200-400MB ในการทำ SSR
3. **ความง่ายในการตรวจสอบและบำรุงรักษา (Maintainability & Debuggability):** แต่ละหน้ามีคู่ไฟล์ชัดเจน (HTML 1 ไฟล์ คู่กับ JS 1 ไฟล์) สามารถเปิดดู Console หรือ Network ใน DevTools ไล่ดูการทำงานได้ทันทีโดยไม่ต้องผ่าน Source Maps ที่ซับซ้อน
4. **การใช้ React เฉพาะส่วนที่จำเป็น (Targeted Micro-bundle):** เราเลือกใช้ React 19 + Recharts เฉพาะส่วนการวาดกราฟสถิติในหน้า `purchase-summary` และ `member-finance` โดยบันเดิลเป็นไฟล์เล็ก ๆ ด้วย `esbuild` ซึ่งทำให้ได้ข้อดีของทั้งสองโลก คือความเบาของ Vanilla และพลังการวาดกราฟของ Recharts ครับ"

Q2 เมื่อไม่มี Redux, Pinia หรือ React Context ระบบจัดการ State ในแต่ละหน้าอย่างไร และป้องกันปัญหา State ไม่ตรงกับหน้าจอ (State Inconsistency) อย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** เข้าใจรูปแบบการออกแบบ State Management ใน Pure JavaScript (Architecture Pattern)

แนวทางการตอบ:

"เราใช้สถาปัตยกรรม **Predictable Single-Source-of-Truth State Management** ครับ โดยในโค้ด JavaScript ของแต่ละหน้า (เช่น `purchases.js`, `contracts.js`, `dividends.js`) จะมีโครงสร้างดังนี้:

- **State Object:** มีตัวแปร `state = { ... }` ประกาศไว้ที่ระดับ Module Scope เป็นแหล่งความจริงจุดเดียว เช่น เก็บข้อมูลรายการ, ตัวกรอง, ค่าค้นหา, และสถานะการโหลด
- **Unidirectional Data Flow (การไหลทิศทางเดียว):** เมื่อเกิด Event จากผู้ใช้ (เช่น พิมพ์ค้นหา, เลือก Combobox) ฟังก์ชัน Event Listener จะไม่ไปไล่แก้ไข DOM กระจัดกระจาย แต่จะไปอัปเดตค่าใน `state` ก่อนเสมอ
- **Centralized Render Function:** เมื่อ `state` เปลี่ยนแปลง จะเรียกฟังก์ชัน `render()` ส่วนกลาง ซึ่งจะอ่านค่าจาก `state` แล้ววาด DOM ใหม่ผ่าน Template Engine ใน `core/dom.js` ทำให้หน้าจอกับข้อมูลในตัวแปรตรงกัน 100% เสมอ ปราศจากปัญหา Desync ครับ"

Q3 กลไกการรวม HTML ด้วย `<!-- @include partials/... -->` ทำงานอย่างไร ทำไมไม่ใช้ Template Engine สำเร็จรูปอย่าง EJS หรือ Pug?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจใน Express Middleware, Regex Parsing และการประมวลผล String ในฝั่ง Server

แนวทางการตอบ:

"กลไกนี้ถูกเขียนขึ้นใน `server/routes/pages.routes.js` ครับ โดยมีหลักการทำงานดังนี้:

1. เมื่อมีคำขอเปิดหน้าเว็บ เซิร์ฟเวอร์จะอ่านไฟล์ HTML จากโฟลเดอร์ `pages/` ด้วย `fs.readFile()`
2. ใช้ Regular Expression ตรวจจับแพทเทิร์น `/<!--\s*@include\s+([a-zA-Z0-9_\-\/.]+)\s*-->/g` แล้วแทนที่ตำแหน่งนั้นด้วยเนื้อหาของไฟล์ partial นั้นจริง เช่น นำ `partials/sidebar.html` มาเสียบ
3. ในโหมด Production ระบบจะทำการ **Cache HTML** ที่ผ่านการประกอบแล้วเก็บไว้ใน Memory ทันที ทำให้คำขอถัดไปส่งไฟล์ออกไปได้เร็วระดับเสี้ยววินาทีโดยไม่ต้องอ่านไฟล์จากดิสก์ซ้ำ
4. **เหตุผลที่ไม่ใช้ EJS/Pug:** เพราะเราต้องการให้หน้า HTML เป็น Pure HTML มาตรฐาน 100% ที่นักพัฒนาสามารถเปิดพรีวิวในเบราว์เซอร์ได้ทันทีโดยไม่ต้องผ่านตัวคอมไพเลอร์ และไม่เสี่ยงต่อช่องโหว่ Code Injection ที่อาจเกิดขึ้นจาก Template Engine ที่รันคำสั่ง JS ได้ครับ"

2 การออกแบบฐานข้อมูลและความสมบูรณ์ของข้อมูล (Database & Integrity)

Q4 ทำไมจึงต้องแยกตาราง `MePair` ออกมาต่างหาก ทำไมไม่เก็บรหัสลูกจ้างไว้ในตาราง Member หรือเก็บรหัสสมาชิกไว้ในตาราง Employee โดยตรง? สอดคล้องกับหลัก Normalization อย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** ความรู้เรื่อง Database Normalization, ความสัมพันธ์ Many-to-Many และ Business Requirement

แนวทางการตอบ:

"เนื่องจากในโลกแห่งความเป็นจริง ความสัมพันธ์ระหว่าง 'สมาชิกเจ้าของสวน' กับ 'ลูกจ้างกรีดยาง' เป็นความสัมพันธ์แบบ **Many-to-Many (N:M)** ครับ:

- สมาชิก 1 คน อาจมีสวนหลายแปลง และจ้างลูกจ้างหลายคนมากรีด
- ลูกจ้าง 1 คน ก็สามารถรับจ้างกรีดยางให้เจ้าของสวนหลายรายในเวลาเดียวกันได้ (เช่น สวน A กรีดวันคี่, สวน B กรีดวันคู่)
- นอกจากนี้ แต่ละคู่สัญญายังมี **สัดส่วนแบ่งรายได้ (memberShare, employeeShare)** และวันหมดอายุของสัญญาที่ไม่เหมือนกัน

หากนำไปเก็บในตาราง Member หรือ Employee จะเกิดปัญหาข้อมูลซ้ำซ้อน (Redundancy) และขัดต่อหลัก **Third Normal Form (3NF)** ทันที การสร้างตาราง `MePair` ขึ้นมาทำหน้าที่เป็น Junction Table ที่เชื่อมโยงด้วย Foreign Key 2 เส้น จึงเป็นการออกแบบที่ถูกต้องตามหลักวิศวกรรมฐานข้อมูลและรองรับการเก็บประวัติการเปลี่ยนสัดส่วนย้อนหลังได้ด้วยครับ"

Q5 อธิบายการทำงานของ Database Transaction ในระบบ โดยเฉพาะในขั้นตอนการรับซื้อน้ำยาง (Purchase) และการเบิกเงิน (Withdrawal) ว่าควบคุมคุณสมบัติ ACID อย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจเรื่อง Data Consistency, Concurrency, ACID Properties และ Prisma API

แนวทางการตอบ:

"ในระบบการเงิน ข้อผิดพลาดที่ยอมรับไม่ได้คือ 'ยอดเงินในกระเป๋าเปลี่ยน แต่ไม่มีบันทึกบิล' หรือ 'มีบิลบันทึก แต่ยอดเงินไม่ขยับ' เราจึงใช้ `prisma.$transaction()` ในการควบคุมความเป็น **Atomic (ทำทั้งหมด หรือไม่ทำเลย)** ครับ:

1. **ในขั้นตอนการรับซื้อ (`createPurchase`):**

```
await prisma.$transaction([
  prisma.purchase.create({ data: purchaseData }),
  prisma.member.update({
    where: { id: memberId },
    data: { walletBalance: { increment: ownerPayout } }
  })
]);
```

   คำสั่งสร้างรายการรับซื้อและการบวกเงินเข้ากระเป๋าของเจ้าของสวนยางจะถูกผูกเข้าด้วยกัน หากเกิดไฟดับหรือ Database Error คำสั่งทั้งคู่จะถูก **Rollback** คืนค่าเดิมทันที

2. **ในขั้นตอนการเบิกเงิน (`createWithdrawal`):** ระบบจะเช็คยอดเงินคงเหลือก่อน และสั่งสร้างเรคอร์ดการเบิกเงิน `W-XXXX` พร้อมสั่ง `decrement` ยอดเงินในกระเป๋าพร้อมกันใน Transaction เดียว ยอดเงินสะสมจึงไม่มีทางติดลบและมีความถูกต้องตามหลักบัญชี 100% ครับ"

Q6 ทำไมในตาราง `Purchase` และ `Withdrawal` จึงต้องเก็บฟิลด์ `ownerPayout`, `employeePayout`, `balanceBefore`, `balanceAfter` ไว้ด้วย ทั้งๆ ที่คำนวณสดจากราคากลางและส่วนแบ่งได้? ขัดกับหลักการลดความซ้ำซ้อนหรือไม่?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจเรื่อง Audit Trail, Historical Snapshot และข้อแตกต่างระหว่าง OLTP กับ Pure Normalization

แนวทางการตอบ:

"นี่คือการออกแบบโดยเจตนาตามรูปแบบ **Financial Audit Snapshot (การเก็บภาพนิ่งทางการเงิน)** ครับ เพราะในระบบธุรกิจจริง:

- **ราคากลางและสัดส่วนสัญญาอาจเปลี่ยนไปตามกาลเวลา:** หากเราคำนวณสดตลอดเวลา วันหนึ่งเมื่อสมาชิกแก้สัญญาจาก 60:40 เป็น 50:50 ยอดเงินของบิลย้อนหลังเมื่อ 6 เดือนก่อนจะเพี้ยนตามไปด้วยทันที
- **ความถูกต้องทางกฎหมายและใบเสร็จ (Immutable Receipt):** ใบเสร็จที่พิมพ์ให้ชาวสวนไปแล้ว ระบุว่าได้รับเงิน 2,500 บาท ข้อมูลในฐานข้อมูลจะต้องบันทึกตัวเลข 2,500 บาทคงที่ไว้ตลอดไปเพื่อใช้เป็นหลักฐานทางบัญชี
- **Audit Trail ของการเบิกเงิน:** การบันทึก `balanceBefore` และ `balanceAfter` ทำให้เราสามารถสอบย้อน (Trace) การเคลื่อนไหวของกระเป๋าเงินทีละรายการได้ทันทีว่าเงินลดลงถูกต้องหรือไม่ โดยไม่ต้องคำนวณใหม่ตั้งแต่บิลแรกของชีวิตสมาชิกครับ"

3 โลจิกทางธุรกิจ การคิดเงิน และการเงินสหกรณ์ (Business Logic)

Q7 ช่วยอธิบายโลจิกในฟังก์ชัน `lookupSeller()` ในหน้ารับซื้อน้ำยางสด ว่าระบบตรวจสอบอย่างไรว่าใครเป็นผู้ส่งยาง และตัดเงินให้ใครบ้าง?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจใน Workflow การรับซื้อน้ำยาง และความสามารถในการไล่โค้ดที่ซับซ้อนใน `server/lib/data/purchases.js`

แนวทางการตอบ:

"ฟังก์ชัน `lookupSeller(code, preferredMemberId)` มีกระบวนการทำงาน 4 ขั้นตอนครับ:

1. **ตรวจจับคำนำหน้า (Prefix Detection):** ระบบรองรับทั้งการพิมพ์ `M-0001`, `E-0001` หรือแม้กระทั่งพิมพ์ตัวเลขเดี่ยว ๆ เช่น `1` ระบบจะมี Regex ช่วยแปลงเป็นรหัสมาตรฐาน
2. **กรณีเป็นสมาชิก (Member):** - ตรวจสอบว่าสมาชิกมีสถานะเป็น `Active` หรือไม่ (ถ้าถูกระงับจะ Reject ทันที)
   - กำหนดว่าเจ้าของสวนมาขายเอง ส่วนแบ่งเจ้าของสวน = 100%, ส่วนของลูกจ้าง = 0%
3. **กรณีเป็นลูกจ้าง (Employee):** - ตรวจสอบว่าลูกจ้างมีสถานะ `Active` หรือไม่
   - ค้นหาสัญญาในตาราง `MePair` ที่มีสถานะ Active และยังไม่หมดอายุ (`contractEndDate == null` หรือยังไม่ถึงกำหนด)
   - **จุดสำคัญ (Multiple Contracts):** หากลูกจ้างรับจ้างให้เจ้าของสวน 3 คน ระบบจะส่งรายชื่อทั้ง 3 สวนกลับมาเป็น Dropdown ให้เจ้าหน้าที่เลือก หรือใช้ `preferredMemberId` ที่เจ้าหน้าที่เลือก เพื่อดึงสัดส่วน `memberShare` และ `employeeShare` ของสวนนั้นมาคำนวณอย่างถูกต้องครับ"

Q8 ระบบจัดการเรื่องเงินปันผลประจำปี (Dividend) อย่างไร มีสูตรคิดอย่างไร และระบบ 'ล็อกหน้าจอข้ามปี พ.ศ.' ทำงานอย่างไรในโค้ด?

> **สิ่งที่กรรมการต้องการวัด:** ตรรกะรอบบัญชีปฏิทินไทย (พ.ศ.), การป้องกันข้อผิดพลาดในการจ่ายเงินปันผล และโค้ดล่าสุดที่เพิ่งอัปเดต

แนวทางการตอบ:

"ระบบเงินปันผลของสหกรณ์คิดตามรอบปีบัญชีปฏิทินไทย (พ.ค. - มี.ค.) โดยมีโลจิกสำคัญดังนี้ครับ:

- **สูตรการคำนวณ:** คิดจาก `∑(dryWeightKg) × rate (บาท/กก.)` โดยระบบจะไปรวบรวมน้ำหนักเนื้อยางแห้งจากทุกบิลรับซื้อของสมาชิกคนนั้นในปี พ.ศ. ดังกล่าวมาคูณกับอัตราปันผลที่กำหนด
- **การตัดเงินเข้ากระเป๋า:** เมื่อกดปุ่ม 'จ่ายเงินปันผล' ระบบจะบวกเงินเข้า `Member.walletBalance` เพื่อให้เบิกเป็นเงินสดได้ และบวกเข้า `Member.dividendBalance` เพื่อเก็บเป็นสถิติสะสม
- **ระบบล็อกหน้าจอข้ามปี พ.ศ. (Dividend Lock System):**

```
// server/lib/data/dividends.js
const currentBuddhistYear = new Date().getUTCFullYear() + 543;
if (buddhistYear > currentBuddhistYear) {
  throw new DividendError("ไม่สามารถคำนวณหรือจ่ายเงินปันผลล่วงหน้าได้");
}
```

ในฝั่ง Client (`dividends.js`) และ Server มีการตรวจสอบว่าหากปี พ.ศ. ปัจจุบันถูกจ่ายไปแล้ว ระบบจะแสดงแถบแบนเนอร์ล็อกสีเทา ปิดปุ่มจ่ายเงิน และซ่อนฟอร์มคำนวณ จนกว่าวันเวลาจริงของเครื่องเซิร์ฟเวอร์จะก้าวข้ามเข้าสู่วันที่ 1 มกราคมของปี พ.ศ. ถัดไปจริง ๆ ระบบจึงจะปลดล็อกและเลือกปีใหม่ให้อัตโนมัติ ป้องกันการกดจ่ายเงินปันผลซ้ำซ้อนอย่างสมบูรณ์ครับ"

4 ความปลอดภัยและการยืนยันตัวตน (Security & Authentication)

Q9 อธิบายระบบ Dual-Session ว่าทำไมระบบจึงต้องแยก Cookie และ Algorithm ระหว่างเจ้าหน้าที่กับสมาชิก? มีการป้องกันการปลอมแปลง Token อย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** ความรู้เชิงลึกเรื่อง Session Security, Cryptographic Signatures, และ Identity Isolation

แนวทางการตอบ:

"เราออกแบบระบบแยกขาดเป็น 2 คุกกี้เซสชัน เพื่อป้องกันการยกระดับสิทธิ์ (Privilege Escalation) ครับ:

มิติการเปรียบเทียบ	Cookie `session` (ฝั่งเจ้าหน้าที่)	Cookie `member_session` (ฝั่งสมาชิก)
เป้าหมาย	เข้าถึงระบบจัดการหลังบ้าน, บัญชี, สัญญา	ดูเฉพาะข้อมูลสวนและกระเป๋าเงินของตนเอง
กุญแจเซ็น Token	`SESSION_SECRET`	`HMAC(SESSION_SECRET, "member-session-v1")`
โครงสร้าง Token	`base64url(JSON Payload) . base64url(HMAC-SHA256 Signature)`	

กลไกป้องกันการปลอมแปลง (Tamper-proofing): ทุกครั้งที่มี Request เข้ามา ฟังก์ชัน `decodeSession()` จะนำ Payload มาคำนวณลายเซ็นใหม่ด้วยกุญแจลับ แล้วเปรียบเทียบกับลายเซ็นที่แนบมาด้วยฟังก์ชัน `crypto.timingSafeEqual()` หากใครพยายามแอบแก้ไข ID หรือ Role ใน Cookie ลายเซ็นจะไม่ตรงกัน และระบบจะตัดสิทธิ์เป็น Unauthorized ทันทีครับ"

Q10 ทำไมรหัสผ่านจึงใช้ `scrypt` แทนที่จะเป็น MD5 หรือ SHA-256? และ `crypto.timingSafeEqual` มีไว้เพื่อป้องกันอะไร?

> **สิ่งที่กรรมการต้องการวัด:** ความรู้เรื่อง Cryptography และการป้องกันการโจมตีทางเวลา (Timing Attacks)

แนวทางการตอบ:

"ในทางความปลอดภัยสารสนเทศ:

- **ทำไมไม่ใช้ MD5/SHA-256:** เพราะเป็น Fast Hash Function ที่ออกแบบมาเพื่อตรวจสอบความสมบูรณ์ของไฟล์ ทำให้แฮกเกอร์สามารถใช้การ์ดจอ (GPU) ยิง Brute-force สุ่มรหัสผ่านได้หลายพันล้านครั้งต่อวินาที รวมถึงมีปัญหา Rainbow Table
- **ทำไมต้องเป็น `scrypt`:** เพราะเป็น **Memory-Hard Function** ที่บังคับให้ต้องใช้ทั้งเวลาและหน่วยความจำ RAM ในการคำนวณ ทำให้การโจมตีด้วย GPU หรือชิป ASIC ทำได้ยากมาก อีกทั้งเรายังสุ่ม Salt ขนาด 16 ไบต์เฉพาะของแต่ละคน ทำให้รหัสผ่านเหมือนกัน แต่ได้ Hash ต่างกัน ป้องกันการเทียบตารางสำเร็จรูปได้ 100%
- **การป้องกัน Timing Attack ด้วย `timingSafeEqual`:** การเปรียบเทียบ String ธรรมดาด้วยเครื่องหมาย `===` ใน JavaScript จะหยุดทำงานทันทีที่เจอตัวอักษรตัวแรกที่ไม่ตรงกัน ทำให้เวลาที่ใช้ในการประมวลผลสั้นยาวไม่เท่ากัน แฮกเกอร์สามารถจับเวลาในระดับไมโครวินาทีเพื่อเดาอักขระทีละตัวได้ แต่ `timingSafeEqual` จะตรวจสอบความยาวและค่าทั้งหมดด้วยระยะเวลาคงที่เสมอ ทำให้ตัดช่องโหว่นี้ได้เด็ดขาดครับ"

Q11 ระบบมีมาตรการป้องกันช่องโหว่ OWASP Top 10 เช่น XSS, SQL Injection และ Open Redirect อย่างไรบ้าง ระบุตำแหน่งในโค้ด?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจด้าน Web Security Practice ในเชิงลึก

แนวทางการตอบ:

1. **SQL Injection:** ป้องกันได้ 100% เนื่องจากทั้งระบบเข้าถึงฐานข้อมูลผ่าน Prisma ORM ซึ่งสร้าง Parameterized Queries เสมอ ไม่มีการต่อสตริงคำสั่ง SQL โดยตรง
2. **Stored / Reflected XSS:** ในฝั่งหน้าบ้าน เรามีกฎเหล็กว่าจะไม่ใช้ `element.innerHTML = data` แต่เราใช้ฟังก์ชัน `setSlot(root, name, text)` ใน `public/js/core/dom.js` ซึ่งกำหนดค่าผ่าน `element.textContent = text` เสมอ ตัวอักษรอย่าง `<script>` จึงถูกเรนเดอร์เป็นตัวอักษรธรรมดา ไม่สามารถรันเป็นสคริปต์ได้
3. **Open Redirect:** ในหน้า Login หากมีพารามิเตอร์ `?next=...` ระบบจะส่งผ่านฟังก์ชัน `sanitizeNext()` ใน `public/js/core/safe-redirect.js` ซึ่งจะตรวจสอบว่า URL ปลายทางต้องเป็น Local Path ในโดเมนเดียวกันเท่านั้น ป้องกันการถูกหลอกให้ Redirect ไปเว็บหลอกลวง (Phishing)
4. **Cookie Hijacking:** คุกกี้ทั้งหมดตั้งค่าแฟล็ก `HttpOnly` ป้องกันไม่ให้สคริปต์หน้าเว็บเข้าถึงได้ และตั้ง `SameSite: "lax"` เพื่อป้องกันการโจมตีแบบ CSRF ครับ"

5 เจาะลึกโค้ดและกลไกหน้าบ้าน (Frontend DOM Engine)

Q12 อธิบายกลไกของฟังก์ชัน `cloneTemplate()`, `setSlot()` และ `action()` ใน `public/js/core/dom.js` ว่าช่วยให้การเขียนโค้ดหน้าบ้านเป็นระบบได้อย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจใน DOM API และการออกแบบ Abstraction Layer สำหรับจัดการ UI

แนวทางการตอบ:

"ไฟล์ `dom.js` ทำหน้าที่เป็นแกนกลางเสมือน Mini-framework ของโปรเจกต์ครับ:

- `cloneTemplate(id)`: ใช้ดึงเนื้อหาจากแท็ก `<template id="...">` ใน HTML ออกมาทำสำเนาด้วย `content.firstElementChild.cloneNode(true)` ซึ่งมีข้อดีคือเบราว์เซอร์จะไม่เรนเดอร์แท็ก Template จนกว่าเราจะสั่ง Clone ทำให้ประหยัดหน่วยความจำ
- `setSlot(root, name, text)`: ค้นหา Element ภายในแม่แบบที่มีแอตทริบิวต์ `data-slot="name"` แล้วยัดข้อความผ่าน `.textContent` นอกจากจะปลอดภัยจาก XSS แล้ว ยังทำหน้าที่แยกโครงสร้างการจัดวาง (HTML) ออกจากตรรกะการป้อนข้อมูล (JS) อย่างชัดเจน
- `action(root, name)`: ค้นหาปุ่มที่มี `data-action="name"` (เช่น view, edit, delete) เพื่อนำมาผูก `addEventListener` ทำให้เราไม่ต้องตั้ง ID ซ้ำซ้อนในตารางที่มีแถวข้อมูลเป็นร้อย ๆ แถวครับ"

Q13 ในคอมโพเนนต์ `debounced-lookup.js` และ `combobox.js` มีปัญหา Race Condition หรือไม่ และโค้ดจัดการอย่างไรหากคำขอค้นหาอันแรกตอบกลับมาช้ากว่าคำขออันหลัง?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจเรื่อง Asynchronous Programming, Network Latency, และ Token-based Request Invalidation

แนวทางการตอบ:

"นี่คือจุดที่เราให้ความสำคัญอย่างมากครับ เพราะหากผู้ใช้พิมพ์ค้นหารหัส `M-0001` แล้วเปลี่ยนใจแก้เป็น `M-0002` อย่างรวดเร็ว หากคำขอของ `M-0001` ตอบกลับมาช้ากว่า อาจทำให้ข้อมูลของ `M-0001` มาทับผลลัพธ์ของ `M-0002` ได้ เราจึงแก้ด้วย **Token Invalidation Pattern** ใน `debounced-lookup.js`:

```
let requestToken = 0;
async function fetchLookup(code) {
  const currentToken = ++requestToken;
  const res = await fetch(buildUrl(code));
  const data = await res.json();

  // ตรวจสอบว่าคำขอนี้ยังเป็นคำขอล่าสุดอยู่หรือไม่
  if (currentToken !== requestToken) {
    return; // ถ้ามีคำขอใหม่กว่าส่งออกไปแล้ว ให้ทิ้งผลลัพธ์นี้ทันที
  }
  applyData(data);
}
```

ทุกครั้งที่มีการพิมพ์ใหม่ ค่า `requestToken` จะถูกบวก 1 เสมอ เมื่อคำขอนั้นตอบกลับมา หากพบว่าตัวเลข Token ในมือไม่ตรงกับค่าล่าสุดในตัวแปรระบบ ผลลัพธ์นั้นจะถูกทิ้งทันที จึงรับประกันว่าหน้าจอจะแสดงข้อมูลของผู้ขายคนล่าสุดที่พิมพ์เสมอครับ"

6 การรองรับข้อผิดพลาด ปัญหา Edge Cases & การขยายระบบ (Scalability)

Q14 หากในอนาคตมีสหกรณ์ขยายสาขา และมีปริมาณการทำรายการรับซื้อยางพร้อมกันมากกว่า 500-1,000 รายการต่อนาที (High Concurrency) ระบบนี้จะเกิดคอขวด (Bottleneck) ที่จุดใด และจะปรับปรุงอย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** วิสัยทัศน์ด้าน System Scaling, Database Connection Pooling และ Architectural Evolution

แนวทางการตอบ:

"หากระบบมีปริมาณการใช้งานสูงขึ้นในระดับพันรายการต่อนาที คอขวดหลักจะอยู่ที่ 2 จุดครับ:

1. **Database Lock Contention ในการอัปเดตกระเป๋าเงิน (`walletBalance`):** ในตาราง `Member` หากสมาชิกคนเดียวกันมีลูกจ้างหลายคนมาส่งยางพร้อมกัน หรือมีการคำนวณเงินสะสม การใช้คำสั่ง `update { increment }` พร้อมกันในหลาย Transaction อาจทำให้เกิด Lock Wait หรือ Deadlock ได้
   แนวทางแก้ไข: ปรับเปลี่ยนระบบบัญชีเป็น **Event-Sourcing / Ledger Model** โดยไม่แก้ยอดคงเหลือในตาราง Member แบบทันที แต่ใช้วิธี Append แถวในตาราง Ledger แล้วให้ Background Worker หรือ Database Trigger มาคำนวณยอดสรุป (Materialized View)
2. **Database Connection Limits:** Prisma Client มีขนาด Connection Pool จำกัด
   แนวทางแก้ไข: ติดตั้ง **PgBouncer** เป็น Connection Pooler ด้านหน้า PostgreSQL เพื่อทำ Transaction Pooling และแยก Read Replica สำหรับคำขออ่านรายงานสถิติ (Purchase Summary) ออกจากเซิร์ฟเวอร์หลักที่ใช้บันทึกธุรกรรมครับ"

Q15 หากอินเทอร์เน็ตที่จุดรับซื้อน้ำยางหน้างานดับกะทันหัน โครงสร้างปัจจุบันรองรับได้หรือไม่? และมีแนวทางในการพัฒนาต่อยอดเป็น Offline-First อย่างไร?

> **สิ่งที่กรรมการต้องการวัด:** ความเข้าใจข้อจำกัดปัจจุบัน (Honest Technical Assessment) และแนวทางพัฒนาในอนาคต

แนวทางการตอบ:

"ในเวอร์ชันปัจจุบัน ระบบเป็นแบบ Online Web Application ซึ่งต้องพึ่งพาการเชื่อมต่อ HTTP API ไปยัง Express Server ดังนั้นหากอินเทอร์เน็ตหลุดจะไม่สามารถบันทึกบิลหรือเบิกเงินได้ครับ
อย่างไรก็ตาม โครงสร้างของโปรเจกต์ที่เป็น Vanilla HTML/JS ได้รับการออกแบบให้พร้อมสำหรับการพัฒนาต่อยอดเป็น **PWA (Progressive Web App) แบบ Offline-First** ได้อย่างง่ายดายในอนาคต โดยมีขั้นตอนดังนี้:

- **Service Worker:** เพิ่มไฟล์ Service Worker เพื่อแคชไฟล์สถิต (HTML, CSS, JS, Fonts) ไว้ในเครื่อง ทำให้เปิดหน้าจอรับซื้อน้ำยางได้แม้ไม่มีเน็ต
- **IndexedDB Storage:** เมื่อบันทึกการรับซื้อยางขณะออฟไลน์ ข้อมูลบิลจะถูกบันทึกไว้ในเบราว์เซอร์ (IndexedDB) ก่อน พร้อมออกใบเสร็จชั่วคราว
- **Background Sync:** เมื่อสัญญาณอินเทอร์เน็ตกลับมา Service Worker จะทำการส่งข้อมูลที่ค้างอยู่ใน IndexedDB ขึ้นไป Sync กับ Express Server และตัดยอดในฐานข้อมูล PostgreSQL อัตโนมัติครับ"